PORA ACADEMY

Working Pandas for Data Science

Content

- Introduction to Pandas
- Installation and Importing of Pandas
- Pandas Data Structures
- Pandas Series
- Pandas DataFrame
- Useful DataFrame Methods
- Understanding Pandas Operations
- Reading and Writing Data in Pandas
- Data Manipulation with Pandas
- Working with Time Series in Pandas
- A Practical Example
- Conclusion

Introduction to Pandas

Pandas: The Python Data Analysis Library

- **Powerful & Versatile** – Simplifies data manipulation in Python.
- **Ideal for Tabular Data** – Works seamlessly with spreadsheets, SQL tables, and structured datasets.
- **Essential for Data Analysis** – The go-to library for statistical analysis and decision-making.
- **Comprehensive Tools** – Supports data cleaning, transformation, and exploration.
- **Seamless Integration** – Works well with NumPy, Matplotlib, and other Python libraries for advanced analysis and visualization.

Installation and Importing of Pandas

- Install Pandas easily via Anaconda, which includes it by default.
- Verify installation with:
  ```
  conda list pandas
  ```
- If missing, install with:
  ```
  conda install pandas
  ```
- Alternatively, install via PyPI:
  ```
  pip install pandas
  ```
- Import Pandas using:
  ```
  import pandas as pd
  ```

Pandas Data Structures

Pandas Data Structures

Pandas provides two types of classes for handling data:

- Series
- DataFrame

Pandas Series

- The series is the object of the Pandas library designed to represent one-dimensional data structures, similar to an array but with some additional features.
- Think of it as a single column or row in a table.

Series	
index	value
0	12
1	-4
2	7
3	9

Declaring a Series

The Series() constructor creates a Pandas Series by passing an array of values as an argument.

Series assume range index (0-n) by default, if index label is not specified.

```
#Creating a Series by passing a list of values
s = pd.Series([1, 3, 5, 6, 8])

#Creating a Series from a dictionary
s= pd.Series({"b": [1,2], "a": 0, "c": 2})

#Creating a Series from a Numpy array
s= pd.Series(np.random.random(5), index=[x for x in 'abcde'])

#Display the array elements
s.values
```

```
array([0.67938161, 0.20148434, 0.46430615, 0.99274192, 0.18419087])
```

```
#Display the index
s.index
```

```
Index(['a', 'b', 'c', 'd', 'e'], dtype='object')
```

Accessing and Modifying Elements in a Series

Selecting Data in a Series

```
ser = pd.Series(range(1,6), index=[x for x in 'abcde'])
# Selecting elements by index position
print(ser.iloc[2])   # Output: 3

# Selecting elements by index label
print(ser['b']) # Output: 2

# Slicing using index range
print(ser[0:2])
print(ser[['b', 'c']])
```

Modifying Elements in a Series

```
# Updating values using index
ser['b'] = 10
print(ser)
```

Operations and Mathematical Functions

- Mathematical functions and other operations such as operators (+, -, *, and /) are applicable to series.
- However, with the NumPy mathematical functions, you must specify the function referenced with numpy

```
# Creating Series from a dictionary
mydict = {'red': 2000, 'blue': 1000, 'yellow': 500}
series_dict = pd.Series(mydict)
print(series_dict)

# Arithmetic operations between Series
mydict2 = {'red': 400, 'yellow': 1000, 'black': 700}
series_dict2 = pd.Series(mydict2)
print(series_dict + series_dict2)

# Applying operations
print(series_dict2 / 2)
print(np.log(series_dict2[series_dict2 > 0]))
```

Pandas Dataframe

- A tabular data structure similar to a spreadsheet
- Extends Series to multiple dimensions
- Acts like a dictionary of Series
- Indexing in DataFrame
  - Row index (labels for each row)
  - Column index (labels for each column)

DataFrame			
	columns		
index	color	object	price
0	blue	ball	1.2
1	green	pen	1.0
2	yellow	pencil	0.6
3	red	paper	0.9
4	white	mug	1.7

Defining a DataFrame

```python
## Using a dictionary of lists

import pandas as pd

data = {'Fruits': ['Apple', 'Grape', 'Pear'], 'Price': [400, 500, 2000], 'Quantity': [5, 10, 6]}

df = pd.DataFrame(data)

df
```

	Fruits	Price	Quantity
0	Apple	400	5
1	Grape	500	10
2	Pear	2000	6

Selecting, Assigning, and Filtering Data

Selecting Data

- Get column names: `df.columns`
- Get index labels: `df.index`
- Access column: `df['price']` or `df.price`
- Access row: `df.loc[2]`

Adding a new column: `df['new'] = [3.0, 1.3, 2.2, 0.8, 1.1]`

Assigning Values

- Adding a new column: `df['new'] = [3.0, 1.3, 2.2, 0.8, 1.1]`
- Deleting a column: `del df['new']`
- Changing a value: `df.at[2, 'price'] = 3.3`

Filtering Data

- Select rows where price < 1.2 `df[df['price'] < 1.2]`

Deleting a Column

- Using `del`:

Selecting Rows using loc and iloc

Selecting Rows using `loc` and `iloc`

- **`loc[]`** – Select rows using labels
- **`iloc[]`** – Select rows using integer position

```
data = {'Fruits': ['Apple', 'Grape', 'Pear'], 'Price': [400, 500, 2000], 'Quantity': [5, 10, 6]}
df = pd.DataFrame(data)
df = df.set_index('Fruits')

df.iloc[2] ## Selects third row
df.loc['Apple'] # Selects row with index 'Apple'
```

DataFrame Arithmetic and Other Operations

- **Apply mathematical functions using NumPy**

```
import numpy as np
np.sqrt(df['price'])
```

- **Adding or subtracting values**

```
df['price'] + 2
df['price'] * 3
```

- **Using Arithmetic Methods**

```
df['Price'].add(2) #Addition
df['Price'].div(2) #Division
df['Price'].mul(1.5) #Multiplication
df['Price'].sub(0.5) #Subtraction
```

- **Reindexing a DataFrame**

```
df_reindexed = df.reindex(['C', 'A', 'B',
'E', 'D']) #Reordering Rows or Columns
df.reindex(['A', 'B', 'C', 'D', 'E', 'F'],
fill_value=0) #Filling Missing Values in
Reindexing
```

- **Transposing a DataFrame**

```
Transposing a DataFrame (
df.T #Swaps rows and columns
```

- **Handling Missing Data**

```
df.isnull() #Checking for Missing Values
df.notnull() #Checking for Missing Values
df.fillna(0) #Filling Missing Values
df.dropna() # Dropping Missing Values
```

Useful DataFrame Methods

- **Exploring & Summarizing Data**

```
df.head(3)  # View first 3 rows
df.tail(3)  # View last 3 rows
df.shape  # Get number of rows and columns (rows, cols)
df.info()  # Display DataFrame summary
df.describe()  # Get summary statistics for numerical columns
```

- **Working with Data**

```
df['column'].unique()  # Find unique values in a column
df['column'].nunique()  # Count unique values in a column
df['column'].value_counts()  # Count occurrences of each unique value
```

- **Sorting & Ordering**

```
df.sort_values('price', ascending=False)  # Sort by 'price' in descending order
df.sort_values(['color', 'price'], ascending=[True, False])  # Sort by multiple columns
```

- **String Operations**

```
df['color'].str.upper()  # Convert text to uppercase
df['color'].str.contains('red')  # Check if 'color' column contains 'red'
```

- **Applying Functions**

```
df['price'].apply(lambda x: x * 2)  # Apply function to each element in 'price'
df.apply(lambda row: row['price'] * 2, axis=1)  # Apply function to each row
```

- **Handling Duplicates**

```
df.duplicated()  # Check for duplicate rows
df.drop_duplicates()  # Remove duplicate rows
```

- **Correlation and Covariance**

```
df.corr()  # Correlation matrix
df.cov()  # Covariance matrix
```

Understanding Pandas Operations

Functions, Sorting & Ranking in Pandas

Applying Functions to Rows/Columns

- Use `apply()` for row/column operations
- Example: Compute range (max - min)

```
f = lambda x: x.max() - x.min()
df.apply(f)  # Column-wise
df.apply(f, axis=1)  # Row-wise
```

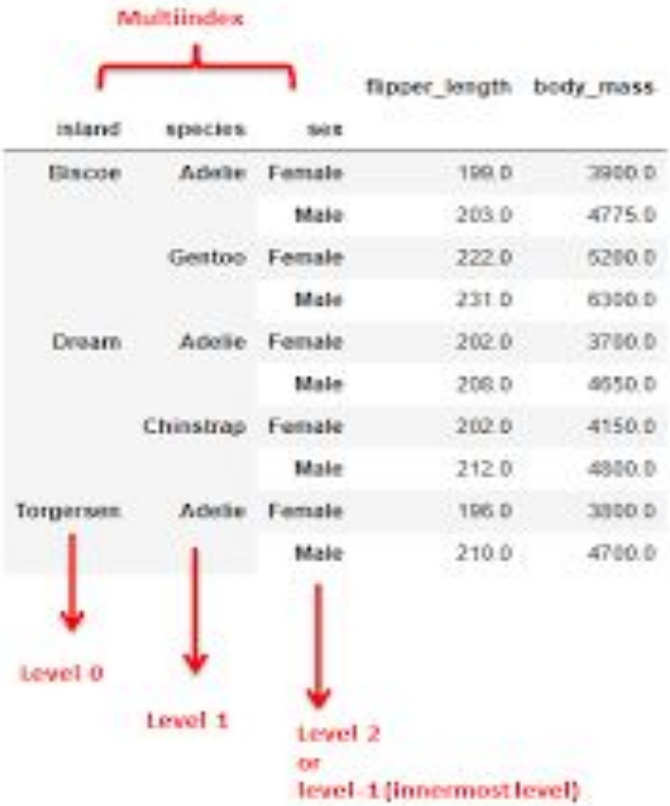
Sorting & Ranking

- `sort_index()` – Sort by index
- `sort_values(by=column)` – Sort by column values
- `rank()` – Assign ranks to values

```
df.sort_values(by='Quantity')
# Sort by column 'Quantity'
df.sort_index()#Sort by index
df.rank()  # Compute ranks
```

Hierarchical Indexing and Multi-Index Aggregation

- Hierarchical indexing in pandas enables multiple index levels on a single axis, facilitating data reshaping and group-based operations like pivot tables.
- The `unstack()` function converts a hierarchically indexed series into a DataFrame by transforming the second index level into columns.



Aggregations with Multi-Index

```
[53] #creating a multi-indexed Series

     multiSer = pd.Series(
         np.random.rand(6),
         index=[['A', 'A', 'B', 'B', 'C', 'C'],
                ['one', 'two', 'one', 'two', 'one', 'two']]
     )
     print(multiSer)
```

```
A  one    0.899329
   two    0.450434
B  one    0.503553
   two    0.942637
C  one    0.017526
   two    0.324702
dtype: float64
```

```
[54] print(multiSer.unstack())
```

```
         one       two
A  0.899329  0.450434
B  0.503553  0.942637
C  0.017526  0.324702
```

```
# Creating a multi-index DataFrame
df = pd.DataFrame(
    np.random.rand(6, 3),
    index=[['X', 'X', 'Y', 'Y', 'Z', 'Z'],
     ['up', 'down', 'up', 'down', 'up', 'down']],
    columns=['A', 'B', 'C']
)

# Summing at the first index level
print(df.groupby(level=0).sum(),"\n\n")

# Taking the mean at the second index level
print(df.groupby(level=1).mean())
```

```
          A         B         C
X  0.990227  0.328648  0.962315
Y  1.375404  0.746595  0.251656
Z  0.231562  0.903289  0.352427


              A         B         C
down  0.536065  0.555661  0.249383
up    0.329665  0.103850  0.272750
```

Reading and Writing Data in Pandas

Pandas File Formats and I/O Functions

- Pandas is a powerful library for data manipulation in Python.
- Reading and writing files is crucial in data science workflows.
- Common file formats handled by Pandas:
  - CSV
  - TXT
  - Excel
  - JSON
  - HTML
  - SQL
  - HDF5
- Pandas provides a comprehensive I/O API for reading and writing data to and from external files

Readers	Writers
read_csv	to_csv
read_excel	to_excel
read_hdf	to_hdf
read_sql	to_sql
read_json	to_json
read_html	to_html
read_stata	to_stata
read_clipboard	to_clipboard
read_pickle	to_pickle
read_msgpack	to_msgpack (experimental)
read_gbq	to_gbq (experimental)

Reading and Writing CSV Files

- **Reading CSV Files:**
  - Use `pd.read_csv('filename.csv')` to load data into a DataFrame.
  - Common parameters include:
    - `sep`: Specify the delimiter (default is comma).
    - `header`: Row number to use as column names.
    - `index_col`: Column(s) to set as index.
    - `na_values`: Additional strings to recognize as NA/NaN.
- **Writing CSV Files:**
  - Use `df.to_csv('filename.csv', index=False)` to export data.
  - Options for encoding, handling missing values, and including/excluding headers.

Example Code:

```
df = pd.read_csv('data.csv', sep=',', header=0)
df.to_csv('exported_data.csv', index=False)
```

Reading and Writing to TXT Files

- **Reading TXT Files:**
  - Use `pd.read_table('filename.txt')` for delimited text files.
  - For fixed-width files, consider `pd.read_fwf('filename.txt')`.

- **Writing TXT Files:**
  - Convert a DataFrame to a text file with `df.to_csv('filename.txt', sep='\t')`
  - Customize delimiters and formatting as required.

Example Code:

```
df = pd.read_table('data.txt', sep='\t')
df.to_csv('output.txt', sep='\t', index=False)
```

Reading and Writing to Json File

- **Reading JSON Files:**
  - Use `pd.read_json('filename.json')` to load JSON data.
  - Handles both simple and nested JSON structures.

- **Writing JSON Files:**
  - Export a DataFrame with `df.to_json('filename.json')`.
  - Adjust parameters such as `orient` (records, columns, index, etc.) to control the output format.

Example Code:

```
df = pd.read_json('data.json')
df.to_json('output.json', orient='records')
```

Reading and writing HTML AND XML Files

- **HTML Files:**
  - Use `pd.read_html('http://example.com')` to extract tables from web pages.
  - Returns a list of DataFrames; choose the one that fits your needs.

- **XML Files:**
  - Use `pd.read_xml('filename.xml')` (requires pandas ≥ 1.3).
  - Allows you to parse XML data into a DataFrame.
  - If XML is complex, additional parameters may be needed to correctly parse elements.

Example Code:

```
# Reading HTML
tables = pd.read_html('http://example.com')
df = tables[0]
# Reading XML (pandas ≥ 1.3 required)
df_xml = pd.read_xml('data.xml')
```

Reading and writing MS EXCEL Files

- **Reading Excel Files:**
  - Use `pd.read_excel('filename.xlsx', sheet_name='Sheet1')` to load data.
  - Supports multiple sheets, custom indices, and column selections.

- **Writing Excel Files:**
  - Use `df.to_excel('output.xlsx', sheet_name='Sheet1', index=False)`
  - Utilize `pd.ExcelWriter` for writing to multiple sheets.

Example Code:

```
df = pd.read_excel('data.xlsx', sheet_name='SalesData')
with pd.ExcelWriter('output.xlsx') as writer:
    df.to_excel(writer, sheet_name='ProcessedData', index=False)
```

Interacting with Databases

- Data is stored in SQL-based or NoSQL databases.
- *pandas.io.sql* provides a unified DB interface via `sqlalchemy`.
- Use *create_engine()* to connect to PostgreSQL, MySQL, Oracle, MSSQL, SQLite.
- Pandas interacts with NoSQL (e.g., MongoDB) via `pymongo`.

```
>>> from sqlalchemy import create_engine
```

For PostgreSQL:

```
>>> engine = create_engine('postgresql://scott:tiger@localhost:5432/mydatabase')
For MySQL
>>> engine = create_engine('mysql+mysqldb://scott:tiger@localhost/foo')
For Oracle
>>> engine = create_engine('oracle://scott:tiger@127.0.0.1:1521/sidname')
For MSSQL
>>> engine = create_engine('mssql+pyodbc://mydsn')
For SQLite
>>> engine = create_engine('sqlite:///foo.db')
```

Data Manipulation with Pandas

Data Preparation

- **Merging:**
  - `pandas.merge()` connects rows using keys (similar to SQL joins).
- **Concatenating:**
  - `pandas.concat()` combines objects along an axis (rows or columns).
- **Combining:**
  - `combine_first()` fills missing values using data from another dataframe.
- **Pivoting:**
  - Rearranges data (e.g., long to wide format) for better analysis.
- **Removing:**
  - Delete columns (`del`) or rows (`drop()`) to clean the dataFrame.

```python
##Merging
import numpy as np
import pandas as pd
frame1 = pd.DataFrame({'id': ['ball', 'pencil', 'pen', 'mug', 'ashtray'],
                       'price': [12.33, 11.44, 33.21, 13.23, 33.62]})
frame2 = pd.DataFrame({'id': ['pencil', 'pencil', 'ball', 'pen'],
                       'color': ['white', 'red', 'red', 'black']})
pd.merge(frame1, frame2)
pd.merge(frame1, frame2, on='id')
pd.merge(frame1, frame2, on='id', how='outer')

#Concatenation:
array1 = np.arange(9).reshape((3, 3))
array2 = np.arange(9).reshape((3, 3)) + 6
np.concatenate([array1, array2], axis=1)

#Combining:
ser1 = pd.Series(np.random.rand(5), index=[1, 2, 3, 4, 5])
ser2 = pd.Series(np.random.rand(4), index=[2, 4, 5, 6])
ser1.combine_first(ser2)
```

Data Transformation

- **Filtering**: selects rows matching condition `df[df['column'] > value]`
- **Selecting** extracts specific columns `df[['col1', 'col2']]`
- **Creating columns**: adds calculated fields `df['new_col'] = df['col1'] + df['col2']`
- **Grouping**: summarizes data `df.groupby('category').agg({'value': 'mean'})`
- **Reshaping**: restructures data `df.pivot(index='date', columns='category', values='amount')`
- **Removing Duplicates**: removes duplicate rows `df.drop_duplicates(subset=['col1', 'col2'], keep='first')`
- **Mapping**: `df['category'] = df['category'].map({'A': 'Group A', 'B': 'Group B'})`
- **Regular Expressions**: extracts patterns `df['text'].str.extract(r'(\d+)', expand=False)`

Data Aggregation

- **GroupBy**: The groupby function is used to group data based on one or more columns and apply aggregation functions to each group.
  - `df.groupby('category')['value'].mean()` calculates average by category
  - `df.groupby(['region', 'category']).sum()` supports multiple grouping levels
- **Pivot Tables**: The pivot_table function provides an easy way to summarize data in a tabular format with more flexibility than groupby. You can set multiple levels for both rows and columns.
  - `pd.pivot_table(df, index='region', columns='category', values='sales', aggfunc='sum')`
- **transform()**: Returns same-sized DataFrame with group calculations
  - `df['pct'] = df.groupby('category')['value'].transform(lambda x: x/x.sum())`
- **apply()** Executes arbitrary functions on each group
  - `df.groupby('category').apply(custom_function)`
- **agg()**: Applies multiple aggregations simultaneously
  - `df.groupby('category').agg({'price': ['min', 'max'], 'qty': 'sum'})`

Working with Time Series in Pandas

Time Series Methods

- **Resampling data**: Resampling allows us to change the frequency of our data.
- **Rolling Calculations**: Rolling calculations are used to calculate moving averages or sums over a specified window. This is particularly helpful for smoothing out short-term fluctuations and observing overall trends.
- **Shifting Data**: Shifting is useful for comparing current values to previous or future values in the time series.
- **pandas.date_range()** creates a DatetimeIndex (essentially a sequence of datetime objects) with regular frequency.
- **period_df.to_timestamp()** converts periods back to timestamps

Time Series Methods

```python
import pandas as pd
import numpy as np

# Generate a date range for two months
date_range =
pd.date_range(start='2023-01-01',
end='2023-02-28', freq='D')

# Generate random sales data
np.random.seed(0)
sales_data = np.random.randint(50, 200,
size=len(date_range))

# Create DataFrame
df = pd.DataFrame({'Date': date_range,
'Sales': sales_data})
df.set_index('Date', inplace=True)
```

```python
# Resampling to get total weekly sales
weekly_sales =
df['Sales'].resample('W').sum()
weekly_sales

# Calculate a 7-day moving average of
sales
df['7-Day Moving Average'] =
df['Sales'].rolling(window=7).mean()
df.head(10)

# Calculate day-over-day change in sales
df['Daily Change'] = df['Sales'] -
df['Sales'].shift(1)
print(df.head(10))
```

A Practical Example

Case Study and Questions

Case Study:

1. A retail company wants to understand sales trends and improve forecasting.
2. Data is collected from various sources: CSV logs, Excel reports, and a SQL database.

Steps to follow:

1. **Data Ingestion:** Read CSV (daily sales) and Excel (monthly targets) files.
2. **Data Cleaning:** Handle missing values and duplicates.
3. **Data Transformation:** Merge datasets and calculate KPIs such as daily sales differences and moving averages.
4. **Visualization:** Plot trends using Matplotlib.

Discussion Questions:

1. How would you merge data coming from different formats?
2. What steps are essential for cleaning the data before analysis?
3. Which pandas functions are most useful for time series analysis in this scenario?
4. How can pivot tables be used to summarize multi-dimensional sales data?

Answers

- **Merging Data:**
  - Use functions like `pd.merge()` and `pd.concat()` to integrate different sources.
- **Data Cleaning:**
  - Key steps include handling missing values (`fillna()`, `dropna()`), removing duplicates (`drop_duplicates()`), and correcting data types.
- **Time Series Analysis:**
  - Functions like `resample()`, `rolling()`, and `shift()` are essential for trend analysis.
- **Pivot Tables:**
  - `pd.pivot_table()` allows summarizing data by multiple dimensions (e.g., region and product category) with custom aggregation functions.

```python
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# Ingest data
sales = pd.read_csv('daily_sales.csv')
targets = pd.read_excel('monthly_targets.xlsx')

# Clean data
sales.fillna(method='ffill', inplace=True)
sales.drop_duplicates(inplace=True)

# Merge datasets
data = pd.merge(sales, targets, on='month', how='left')

# Calculate moving average
data['7-Day MA'] = data['sales'].rolling(window=7).mean()

# Plotting
plt.plot(data['date'], data['sales'],
         label='Daily Sales')
plt.plot(data['date'], data['7-Day MA'],
         label='7-Day Moving Average')
plt.legend()
plt.show()
```

Conclusion

Key Takeaways:

- Pandas is a versatile library that simplifies data manipulation and analysis.
- Mastering its I/O capabilities, data cleaning, transformation, and time series functions is crucial for any data scientist.

Final Thoughts:

- Practical application through case studies reinforces the concepts learned.
- Continuous learning and exploration of pandas' extensive functions will enhance your data analysis workflow.

Any Questions

Reference

Fuchs, F. (n.d.) *Python Data Analytics: With Pandas, NumPy, and Matplotlib*. Available at: https://www.pdfdrive.com/python-data-analytics-with-pandas-numpy-and-matplotlib-e183971250.html (Accessed: 21 March 2025).

The Pandas Development Team (n.d.) *Pandas documentation*. Available at: https://pandas.pydata.org/docs/ (Accessed: 21 March 2025).

PORA
ACADEMY

THANK YOU!